

Harness GitOps PR Pipelines and ApplicationSets

1. Introduction to Harness GitOps PR Pipelines

Harness is a software delivery platform that provides tools to automate application delivery, deployment workflows, and operational processes.

In the workflow described by the Harness documentation, a PR Pipeline coordinates changes to a GitOps repository, pull-request review and merging, GitOps synchronization, and optional post-deployment verification.

Instead of modifying a Kubernetes cluster directly, the workflow proposes a change through Git.

The pull request provides a reviewable change, while the GitOps controller reconciles the cluster with the desired state represented in the repository.

Why use a PR Pipeline?

Suppose an organization manages development, staging, and production environments.

Each environment may have different configuration requirements, such as replica counts, resource limits, or application-specific settings.

Managing these environments manually can introduce configuration inconsistencies and require repetitive work.

A PR Pipeline helps automate this process.

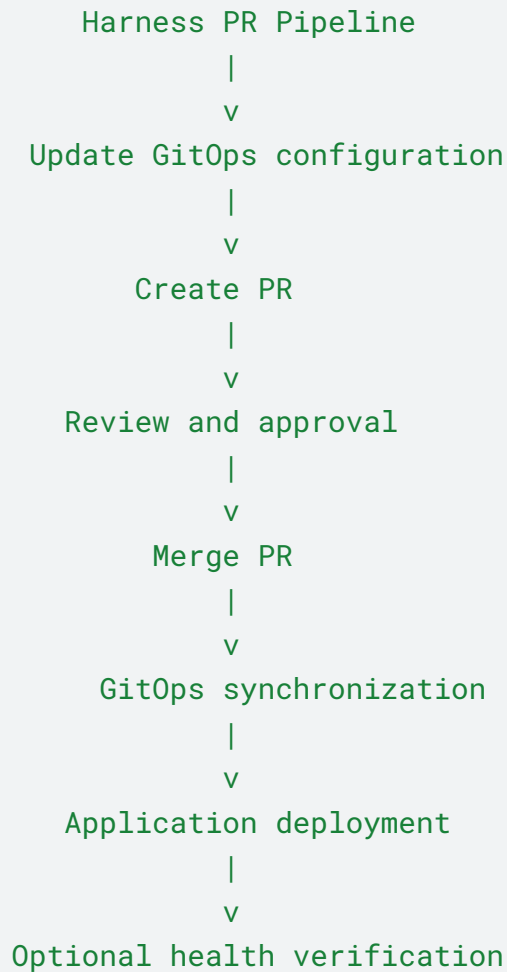
The general workflow is:

None

Developer requests a configuration change

|

v



Main benefits

- **Automation:** Reduces repetitive configuration and deployment tasks.
- **Consistency:** Reuses established configuration and application templates.
- **Auditability:** Git records configuration changes and their history.
- **Governance:** Pull requests provide a review and approval boundary.
- **Scalability:** Supports workflows that create or update applications across multiple environments.
- **Observability:** Additional verification steps can evaluate deployment health.

2. Understanding the main components

Before studying the complete workflow, it is important to understand the main technologies and components involved.

Component	Purpose
Harness	Orchestrates pipeline execution and delivery workflows.
GitOps repository	Stores the desired configuration for applications and environments.
Pull Request (PR)	Proposes a configuration change for review before merging.
Update Release Repo	Harness pipeline step used to update release-repository configuration and create a PR.
Argo CD	Reconciles applications with their desired state in Git.
Argo CD Application	Defines an application managed by Argo CD.
ApplicationSet	Generates Argo CD Application resources from templates and generator data.
Git generator	Reads matching repository files or paths and supplies parameters to ApplicationSet templates.
GitOps Sync	Harness pipeline step used to synchronize specified GitOps applications.
Fetch Linked Apps	Harness pipeline step used in the documented scenario to retrieve information about linked applications.
Continuous Verification (CV)	Evaluates monitored metrics after deployment.
Datadog	Monitoring and health source used in the supplied CV example.

Important distinction: Harness orchestrates the pipeline, ApplicationSet generates Argo CD Application resources, and Argo CD manages the desired state of the resources associated with those applications.

These components have different responsibilities.

3. Understanding Argo CD ApplicationSets

3.1 What is an ApplicationSet?

An **Argo CD ApplicationSet** is a feature that automates the creation and management of multiple Argo CD Applications from a single configuration.

Normally, a platform engineer could create an Argo CD Application resource manually for each environment.

For example:

```
None
Development Environment
    |
    v
Argo CD Application: development

Staging Environment
    |
    v
Argo CD Application: staging

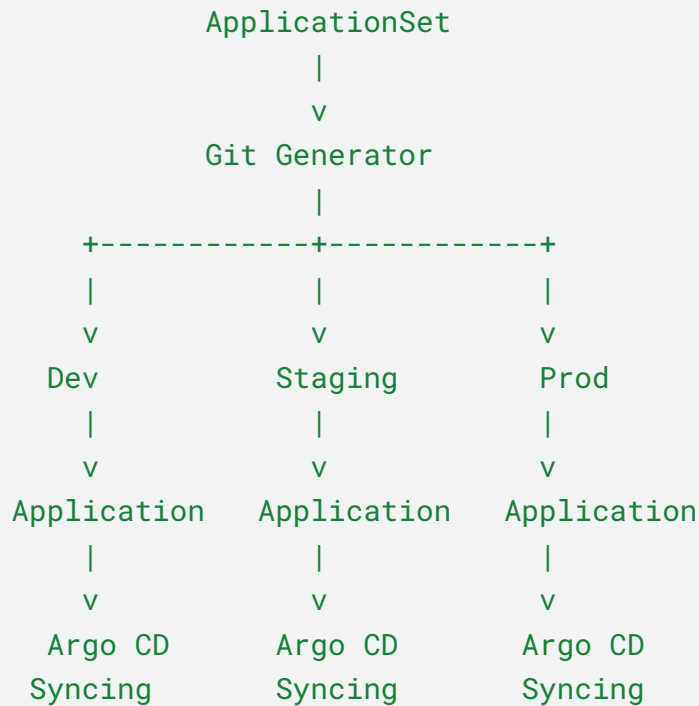
Production Environment
    |
    v
Argo CD Application: production
```

This approach requires maintaining multiple Application definitions.

ApplicationSet provides a way to generate these Application resources automatically using a template and data from a configured generator.

For example:

None



The actual behavior depends on the ApplicationSet generator, template, repository configuration, and reconciliation process.

3.2 How does ApplicationSet work?

ApplicationSet uses two important elements.

1. Generator

The generator discovers or produces the data used to create applications.

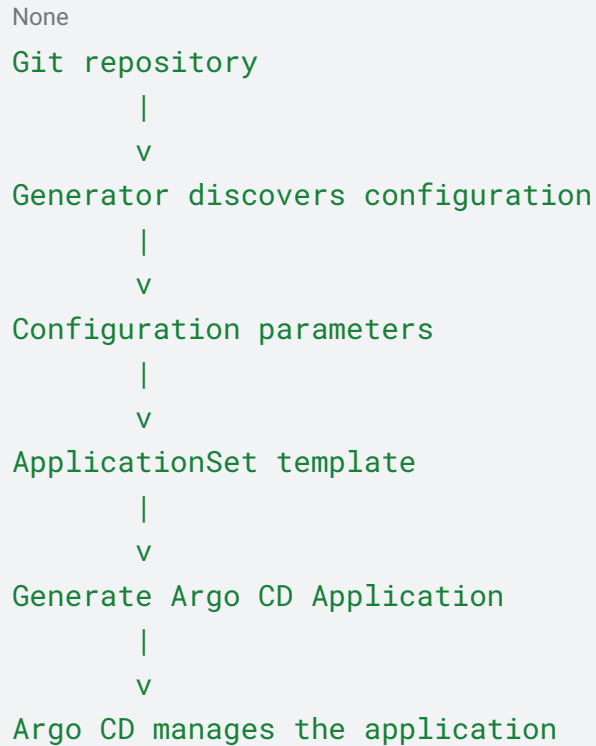
In the example from the Harness documentation, the Git generator discovers environment configuration files in a repository.

2. Template

The template defines how each generated Argo CD Application should look.

The generator provides parameters, and the template uses those parameters to generate the Application resources.

The process is:



3.3 Benefits of ApplicationSets

Consistency

Applications are generated from a common template, helping teams maintain consistent definitions.

Reduced manual effort

Platform engineers do not need to create every Application resource individually.

Configuration separation

Environment-specific configuration can be stored separately from the shared application template.

Auditability

Git records changes to the configuration files, allowing teams to review the history.

Dynamic application creation

When a new matching configuration is added to the repository, ApplicationSet can generate another Application during reconciliation.

Centralized management

Multiple environment configurations can be maintained in a single repository or an organized repository structure.

4. Understanding ApplicationSet generators

The Harness documentation lists several generator types supported by Harness GitOps.

These include:

1. Git Generator
2. List Generator
3. Cluster Generator
4. Matrix Generator
5. Merge Generator
6. SCM Provider Generator
7. Pull Request Generator
8. Cluster Decision Resource Generator
9. Plugin Generator

Each generator provides data differently.

For example, a List Generator can provide predefined parameter values, while a Cluster Generator can use cluster-related information.

However, the Harness ApplicationSet PR Pipeline example focuses on the **Git Generator**.

Official reference:

[Argo CD ApplicationSet Generator Documentation](#)

5. Understanding the Git Generator

5.1 What is the Git generator?

The Git generator dynamically creates parameters for Argo CD Application resources by reading information from a Git repository.

The Harness documentation highlights the file-oriented Git generator, particularly its use of `config.json` files.

These files contain configuration information that can be supplied to an ApplicationSet template.

Instead of creating a separate application definition manually for every environment, the team defines a common template and supplies environment-specific configuration.

Example repository structure

Consider a repository organized like this:

```
None
gitops-config/
|
├── environments/
|   |
|   ├── development/
|   |   └── config.json
|   |
|   ├── staging/
|   |   └── config.json
|   |
|   └── production/
|       └── config.json
|
└── applications/
```

Each environment has its own configuration file.

An illustrative development configuration could look like this:

```
None
{
  "environment": "development",
```

```
"replicaCount": 1,  
"region": "development"  
}
```

A staging configuration could contain different values:

```
None  
{  
  "environment": "staging",  
  "replicaCount": 2,  
  "region": "staging"  
}
```

These are simplified examples. The actual configuration schema depends on the ApplicationSet template and the application configuration that consumes the generated values.

5.2 How does it generate applications?

The workflow is:

```
None  
Git repository  
  |  
  v  
Git generator scans configured paths  
  |  
  v  
Find matching configuration files  
  |  
  v  
Extract configuration parameters  
  |  
  v  
Apply ApplicationSet template  
  |  
  v
```

Generate Argo CD Applications

The template determines how the discovered parameters are used.

For example, a shared template might generate an Application for each environment and use the environment name to determine the application name or configuration path.

5.3 Benefits of file-based configuration

Arbitrary configuration values

The file-oriented generator can expose configuration keys as parameters for the ApplicationSet template.

The template must still be configured to use those values correctly.

Decoupling configuration from templates

The shared template can remain consistent while each environment has its own configuration.

For example, development and production may use different replica counts without requiring separate copies of the entire template.

Version control and auditability

Every committed configuration change becomes part of the repository's history.

Reviewers can examine changes before they are merged.

Dynamic application generation

A new matching configuration file can lead to another generated Application when ApplicationSet reconciles the repository.

Centralized management

Teams can organize environment configurations in a common repository and use pull requests to manage changes.

6. Understanding the Harness PR Pipeline workflow

The Harness documentation describes a scenario in which a team wants to provision a new environment with a unique configuration.

The workflow uses four main activities:

1. Update Release Repo
2. Review and merge the PR
3. GitOps Sync
4. Fetch Linked Apps

These steps connect the PR workflow with the ApplicationSet mechanism.

6.1 Step 1 — Update Release Repo

The **Update Release Repo** step creates or updates the required configuration in the release repository.

In the ApplicationSet scenario, this involves creating a new environment configuration file or updating an existing one.

For example:

```
None
environments/
  development/config.json
  staging/config.json
```

A developer requests a new environment named `test-east`.

The pipeline generates the required configuration:

```
None
environments/
  development/config.json
  staging/config.json
  test-east/config.json
```

The new file provides the data needed by the Git generator.

The Update Release Repo step then opens a pull request for the change.

The important outcome is that a proposed configuration change has been created in Git. It does not mean the environment is already deployed.

6.2 Step 2 — Review and merge the PR

The Harness documentation explains that the pipeline can wait for the PR to be reviewed and merged through the Git provider.

To use this workflow, enable **Wait for PR merge** on the Update Release Repo step.

The process becomes:



The Git provider's PR review and merge process acts as the deployment gate.

According to the supplied documentation, this configuration does not require a separate Harness Approval or Merge PR step for this particular workflow.

This is specific to the documented configuration. Other Harness workflows can use explicit approval and merge steps.

6.3 Step 3 — GitOps Sync

After the PR is merged, the pipeline executes the GitOps Sync step.

The purpose is to synchronize the relevant GitOps application or applications.

In the ApplicationSet scenario, the updated repository configuration becomes available to ApplicationSet reconciliation.

ApplicationSet can then detect the new configuration and generate the corresponding Argo CD Application.

It is important to distinguish two operations:

ApplicationSet reconciliation

```
None
New configuration
  |
  v
ApplicationSet detects configuration
  |
  v
Generate or update Application resource
```

Argo CD application synchronization

```
None
Generated or updated Application
  |
  v
Argo CD compares desired and actual state
  |
  v
Synchronize managed resources
```

These operations are related but not identical. Their timing depends on controller reconciliation and pipeline configuration.

6.4 Step 4 — Fetch Linked Apps

The final step in the documented scenario is Fetch Linked Apps.

The purpose is to retrieve information about the application created or linked by the ApplicationSet workflow.

This is useful because the new application might not have existed when the pipeline was initially configured.

The pipeline can use this step to check the application associated with the new environment.

The supplied excerpt does not provide the complete input schema or output fields for this step, so those details should be checked against the Harness version being used.

7. Complete ApplicationSet PR Pipeline example

Consider an organization that already operates development and staging environments.

The team now wants to create a new testing environment.

The desired workflow is:

Step 1: Create the configuration

The new file is:

```
None
environments/test-east/config.json
```

An illustrative configuration could be:

```
None
{
  "environment": "test-east",
  "replicaCount": 2,
  "region": "east"
}
```

The fields are examples rather than a mandatory Harness schema.

Step 2: Create the PR

Harness updates the release repository and creates a PR.

The PR proposes the new environment configuration.

Step 3: Review and merge

The team reviews the configuration and merges the PR through the Git provider.

With Wait for PR merge enabled, the pipeline waits for the merge before continuing.

Step 4: Run GitOps Sync

Harness runs the configured GitOps Sync step.

The ApplicationSet controller can then discover the new configuration during reconciliation.

Step 5: Generate the application

ApplicationSet uses the Git generator and its template to create the corresponding Argo CD Application.

Step 6: Retrieve the linked application

Fetch Linked Apps retrieves information about the application associated with the new environment.

Step 7: Verify deployment health

An optional Continuous Verification step can evaluate monitoring data after synchronization.

The complete workflow is:

None

Developer requests test-east environment

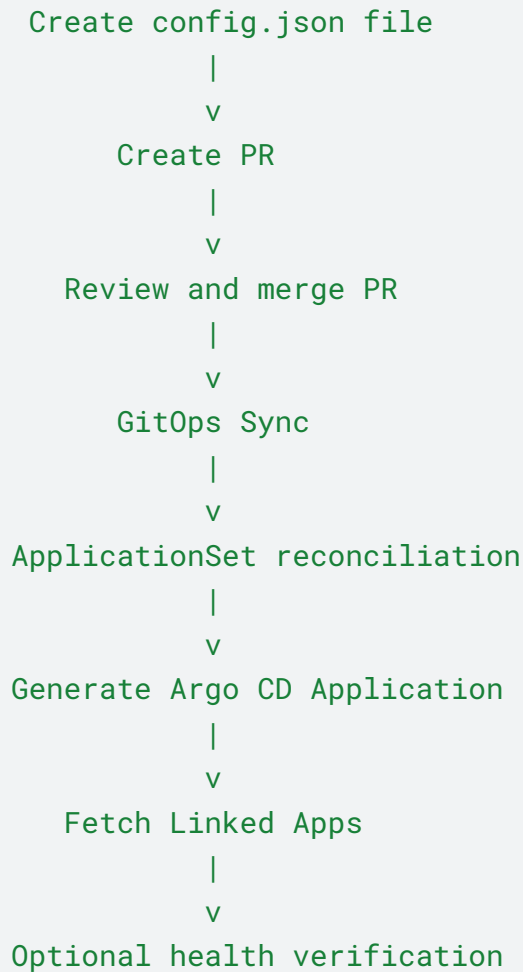
|

v

Update Release Repo

|

v



The PR merge, ApplicationSet reconciliation, and workload synchronization are distinct events. A successful PR merge should not automatically be treated as proof that the new environment is healthy.

8. Continuous Verification using Datadog

The second part of the documentation explains how to add a **Continuous Verification (CV)** step to an existing GitOps PR Pipeline.

The example uses Datadog to monitor Kubernetes deployment replica metrics.

8.1 Why is Continuous Verification needed?

GitOps synchronization and application health are different concerns.

A deployment might synchronize successfully but still have operational problems.

For example:

- The expected number of replicas may not be available.
- The application may have errors.
- The workload may not be behaving as expected.

Continuous Verification adds a way to evaluate monitored metrics after deployment.

The documented example focuses on Kubernetes replica metrics.

8.2 Prerequisites

According to the supplied documentation, you should first complete the prerequisites in the Harness Community GitOps Samples repository and create the GitOps PR Pipeline.

References:

- [PR Pipeline prerequisites](#)
- [Creating a PR Pipeline](#)

8.3 Add the Verify step

The documented procedure is:

1. Click **+ Add Step**.
2. Under **Continuous Verification**, select **Verify**.
3. Provide a name for the Verify step.
4. Choose the Continuous Verification type that matches the deployment strategy.
5. Under **Health Sources**, click **+ Add**.
6. Select **Datadog** as the health source type.
7. Provide a health source name.
8. Configure a Datadog connector.
9. Select an existing Datadog dashboard or choose **+ Manually input query**.
10. Submit the configuration to open the Query Specification page.

For Datadog connector configuration, see the [Harness Datadog health source documentation](#).

8.4 Configure the metric

The supplied example uses this metric name:

None

```
kubernetes_state.deployment.replicas_available
```

It also gives the following manually entered query:

None

```
avg:kubernetes_state.deployment.replicas{kube_app_instance:gitops-sample-application}
```

The example tag is:

None

```
kube_app_instance:gitops-sample-application
```

The tag is used to scope the query to the relevant application instance.

Important detail: The metric name listed in the documentation and the metric used in the example query are different. The former is

`kubernetes_state.deployment.replicas_available`, while the query uses `kubernetes_state.deployment.replicas`.

Do not assume they are interchangeable. Confirm which metric is available and appropriate in your Datadog environment before configuring the pipeline.

8.5 Configure the Service Instance Identifier

The supplied guide explains that selecting Continuous Verification for a service requires a **Service Instance Identifier (SII)**.

The SII identifies individual instances of a service within the infrastructure.

The example is:

None

```
kube_app_instance
```

This allows the verification configuration to associate the metric with the relevant service instance.

The identifier must correspond to the telemetry tags and service configuration available in the actual environment.

8.6 Run the pipeline

After configuration, run the pipeline.

The Verify step displays a graph based on the configured health source and query.

The supplied YAML example includes:

Setting	Example value
Verification duration	10m
Sensitivity	HIGH
Baseline	LAST
Fail on no analysis	true
Verification failure handling	Manual intervention
Unknown error handling	Manual intervention

These are settings from the supplied example, not universal defaults that are appropriate for every application.

They should be adjusted according to the application, monitoring signals, rollout strategy, and operational requirements.

8.7 What can replica metrics tell you?

Replica metrics help indicate whether the expected workload replicas are available.

However, replica availability alone does not prove that the application is functioning correctly.

A more comprehensive verification strategy may also use application latency, error rates, logs, synthetic checks, or business-level metrics.

Those are additional design options, not features demonstrated in the supplied example.

9. Understanding the supplied Harness pipeline YAML

The supplied YAML describes a Deployment stage with GitOps enabled.

The execution steps are:

```
None
GitOpsUpdateReleaseRepo
    |
    v
HarnessApproval
    |
    v
MergePR
    |
    v
GitOpsSync
    |
    v
Verify
```

This is the sequence in the supplied YAML example.

It is worth noting that this sequence differs from the ApplicationSet guide's workflow, which can use Wait for PR merge on Update Release Repo instead of separate Harness Approval and Merge PR steps.

9.1 Update Release Repo

The YAML contains a step of type:

```
None
type: GitOpsUpdateReleaseRepo
```

Its name is:

None

```
name: Update Release Repo
```

It also defines a `replicaCount` input variable and a PR title:

None

```
variables:
  - name: replicaCount
    type: String
    value: <+input>
prTitle: Update values.yaml
```

The step updates the release repository using the configured pipeline inputs and creates the corresponding PR.

The exact effect of the variable depends on the service and step configuration.

9.2 Harness Approval

The example contains a Harness Approval step.

Its configuration includes:

- An approval message.
- An approver user group.
- A minimum approver count of one.
- A timeout of one day.

This is an explicit Harness-controlled approval gate.

It differs from a workflow where the Git provider's PR review and merge process serves as the gate.

9.3 Merge PR

The example includes:

None

```
type: MergePR
```

It also contains:

None

```
deleteSourceBranch: true
```

The step merges the PR as part of the pipeline workflow, subject to configured permissions and conditions.

Do not combine this step with a separate wait-for-merge configuration without considering the intended sequence. The supplied ApplicationSet documentation presents waiting for the Git-provider merge as an alternative to separate approval and merge steps for that scenario.

9.4 GitOps Sync

The supplied example identifies the application:

None

```
applicationsList:  
  - applicationName: gitops-sample-application  
    agentId: gitopspragent
```

It also includes:

None

```
prune: false  
dryRun: false  
applyOnly: false  
forceApply: false
```

And:

None

```
retry: false
```

The sync step is configured with a timeout of ten minutes.

These are example values, not universal recommendations.

In particular, `prune: false` means the sample does not request pruning through that setting. Changes to pruning or force-apply behavior should be reviewed carefully because they can affect resources managed by GitOps.

9.5 Verify

The supplied YAML contains a Verify step configured with automatic verification.

It specifies:

- A ten-minute verification duration.
- High sensitivity.
- The last execution as the baseline.
- Failure handling for verification errors.
- Failure handling for unknown errors.

The YAML excerpt does not contain the complete Datadog health-source and query configuration.

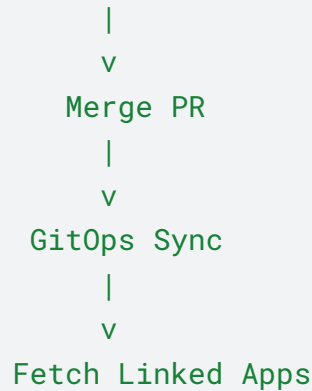
Therefore, the YAML example and the Datadog setup instructions should not be assumed to constitute a fully configured pipeline simply by combining the two excerpts.

10. Two ways to control PR approval and merging

The supplied materials describe two different patterns.

Pattern A: Git-provider merge acts as the gate





This pattern is suitable when the Git provider's required reviews, status checks, and branch-protection rules are the authoritative approval mechanism.

The Update Release Repo step must be configured to wait for the merge.

Pattern B: Harness explicitly controls approval and merge



This pattern makes approval and merging explicit pipeline steps, as shown in the supplied YAML.

Which pattern should be used?

The decision depends on the organization's governance model.

For example, an organization may prefer to manage approvals entirely through GitHub or GitLab. Another may require an explicit Harness approval step before the pipeline is permitted to merge the PR.

Avoid unintentionally duplicating approval gates or creating a sequence in which synchronization begins before the required merge has occurred.

11. Failure handling and troubleshooting

A reliable GitOps pipeline should distinguish PR failures, synchronization failures, and health-verification failures.

11.1 PR creation or merge problems

Possible causes include:

- Incorrect repository permissions.
- Missing Git-provider credentials.
- Branch-protection requirements.
- Failed status checks.
- Invalid configuration files.
- A PR that has not yet been approved or merged.

The pipeline should make clear whether it failed to create the PR, is waiting for review, or was blocked by a repository policy.

11.2 ApplicationSet does not generate the expected Application

Potential areas to investigate include:

- The configuration file does not match the Git generator's configured paths.
- The repository or revision is incorrect.

- The JSON file is invalid.
- Required configuration fields are missing.
- ApplicationSet has not yet reconciled the repository.
- The template or its permissions are incorrect.

These are general troubleshooting possibilities rather than a diagnosis of any specific environment.

11.3 GitOps synchronization fails

If the Application is out of sync or fails to apply the desired resources, investigate:

- The Argo CD Application status.
- Synchronization errors.
- Manifest validity.
- Kubernetes events.
- Controller logs.
- Resource permissions and dependencies.

11.4 Continuous Verification fails

A verification failure could indicate:

- A monitored metric crossed a configured threshold.
- Analysis could not complete.
- Required telemetry was unavailable.
- The query or service-instance tag was incorrect.

The failure strategy should help distinguish actual application-health problems from missing or invalid telemetry.

11.5 Safe handling of destructive changes

The supplied YAML uses:

```
None
prune: false
forceApply: false
```

These settings do not replace careful review of manifests, permissions, and resource ownership.

Any pipeline that permits deletion, pruning, or force application should have appropriate review requirements and recovery procedures.

12. Security and governance considerations

The supplied excerpts do not describe the complete Harness security architecture. The following are general considerations for a PR-driven GitOps workflow.

Least-privilege permissions

Give the pipeline only the repository and cluster permissions it needs.

Secret management

Do not store credentials or access tokens in environment configuration files committed to Git.

Branch protection

Require reviews and appropriate status checks on protected branches.

Separation of duties

Decide whether the person creating a PR can approve their own change.

Audit trail

Where possible, retain the relationship between:

None

Pipeline execution

|

v

Pull request

|

v

Merged commit

```

      |
      v
GitOps synchronization
      |
      v
Verification result

```

Environment protection

Production changes may require stronger approval requirements than development changes.

Change traceability

Include enough information to identify which environment and application a PR affects.

Monitoring accuracy

Ensure that Datadog tags identify the intended application and that queries do not accidentally aggregate unrelated workloads.

These are recommended design considerations; the supplied documentation does not establish that every control is enabled automatically.

13. Plural PR Automation vs. Harness PR Pipelines

You previously studied Plural PR Automation, which uses a `PrAutomation` Kubernetes custom resource, configuration inputs, Liquid templates, an SCM connection, and generated pull requests.

Harness uses a different workflow model.

The following table compares the two approaches based on the previously reviewed Plural material and the Harness documentation supplied here.

Capability	Plural PR Automation	Harness PR Pipelines
------------	----------------------	----------------------

Main abstraction	PrAutomation resource and templates	Harness pipeline and configured steps
Developer input	Inputs defined by the PR automation	Pipeline inputs and release configuration
File generation	Liquid templates and create/update definitions	Update Release Repo step and repository configuration
PR creation	SCM connection and PR automation	Update Release Repo step
PR approval	PR review and associated governance workflows	Git-provider merge gate or explicit Harness approval and merge steps
Dynamic Argo CD Application creation	Not the central mechanism in the reviewed Plural example	ApplicationSet with a Git generator
GitOps synchronization	GitOps controller processes merged repository state	Harness GitOps Sync step coordinates synchronization, while Argo CD reconciles the Application
Post-deployment verification	Depends on the configured workflow and integrations	Explicit Verify step with a health source such as Datadog
PR-to-deployment tracking	Plural includes PR correlation and related service lifecycle features	Pipeline results and linked-application/verification steps; complete correlation depends on configuration
Main architectural emphasis	Self-service PR automation within Plural's platform model	Orchestrated release-repository changes, GitOps delivery, and optional verification

Main difference

Plural emphasizes defining reusable self-service PR automations as resources that developers can invoke.

Harness emphasizes coordinating the steps of a delivery workflow through pipelines.

ApplicationSet adds another capability to the Harness example: it can dynamically generate Argo CD Applications from configuration files discovered in Git.

This is a comparison of the reviewed examples, not a comprehensive product, licensing, or feature comparison.

14. Relevance to WSO2 Project 702: PR-Driven Self-Service GitOps for OpenChoreo Developers

Your Project 702 focuses on making OpenChoreo resource changes through a developer portal and Git pull requests rather than relying only on direct cluster operations.

The proposed architecture includes:

- Backstage developer portal and scaffolder.
- `occ` for generating OpenChoreo manifests.
- Configurable GitOps repository paths.
- GitHub PR creation.
- Conflict detection.
- Pending-change status.
- Flux-based synchronization and OpenChoreo reconciliation.

The Harness documentation offers several useful design patterns for this project.

14.1 Mapping Harness concepts to OpenChoreo

Harness concept

Potential OpenChoreo equivalent

Update Release Repo

Backstage scaffolder action generates or updates GitOps repository content.

Pipeline inputs	Backstage scaffolder form collects developer intent.
PR creation	Backstage GitHub integration opens a PR.
PR review and merge	GitHub review and branch-protection rules gate the change.
GitOps Sync	Flux reconciles the merged repository state.
Fetch Linked Apps	A portal status component retrieves the affected OpenChoreo resource or environment status.
Continuous Verification	A future extension could display deployment health or telemetry after reconciliation.
ApplicationSet Git generator	Relevant only if the architecture uses Argo CD ApplicationSets to generate Argo CD Applications from repository configurations.

The mapping is conceptual. Harness pipeline steps are not automatically available in Backstage or OpenChoreo.

14.2 What can Project 702 learn from Harness?

A. Use the PR as an approval boundary

A developer should be able to request a change through the portal.

The portal generates the appropriate configuration and opens a PR. Reviewers can inspect the change before it is merged.

This aligns with the central objective of Project 702.

B. Separate PR merge from deployment status

A merged PR does not necessarily mean the application is running.

The portal should distinguish between different stages of the workflow.

For example:

None

Request submitted

|

v

Configuration generated

|

v

PR opened

|

v

PR merged

|

v

GitOps reconciliation pending

|

v

Resource reconciled

|

v

Runtime health checked

Not every state needs to be implemented in the first version. The important point is to represent states the system can reliably observe.

C. Return useful information after PR creation

A self-service action should provide the PR URL and a clear status indicating that the change is awaiting review.

Where useful, it should also identify the target repository, branch, and affected resource.

D. Connect the PR to the affected resource

The Fetch Linked Apps concept suggests that a workflow should identify the resource created or updated by a PR.

For OpenChoreo, the relevant resource could be a Workload, ComponentRelease, ReleaseBinding, or environment, depending on the action.

E. Consider verification after the core workflow works

Project 702 should first establish a reliable end-to-end flow:

```
None
Portal action
  |
  v
Generate OpenChoreo configuration
  |
  v
Create PR
  |
  v
Merge PR
  |
  v
GitOps reconciliation
  |
  v
Observe resource status
```

Deployment-health verification can be evaluated as a later extension.

14.3 What should not be copied directly?

Do not introduce ApplicationSets automatically.

Your Project 702 proposal initially focuses on Flux and OpenChoreo's existing GitOps model. ApplicationSet is an Argo CD feature and is not automatically required for that architecture.

Do not copy Harness-specific pipeline steps into Backstage.

Backstage scaffolder actions and integrations would need to implement the equivalent behavior.

Do not bypass GitOps reconciliation.

The merged repository state should remain the source of truth. The portal should not treat a direct imperative deployment as a substitute for the intended GitOps workflow.

Do not use Datadog verification as a replacement for resource-status checks.

Monitoring metrics and GitOps reconciliation status answer different questions.

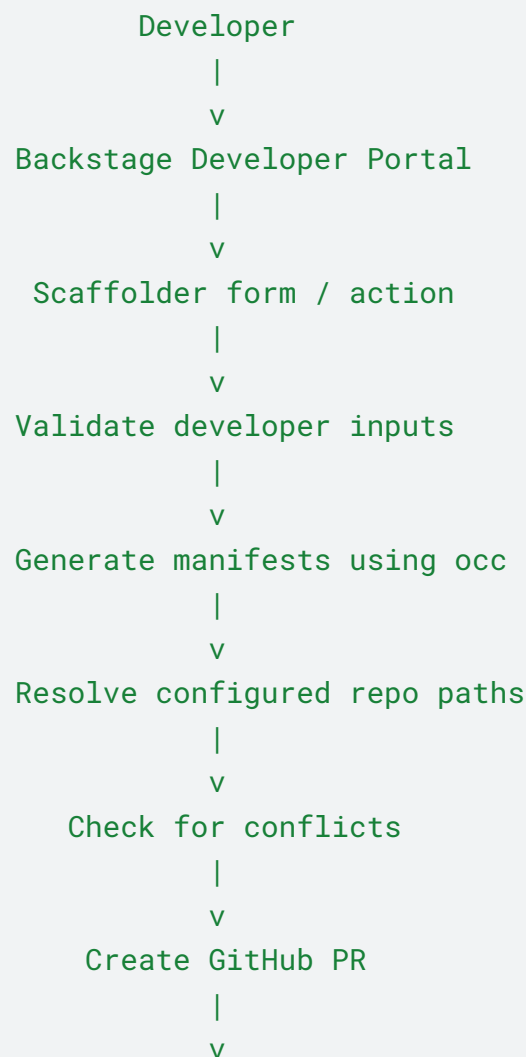
Do not silently resolve conflicting PR changes.

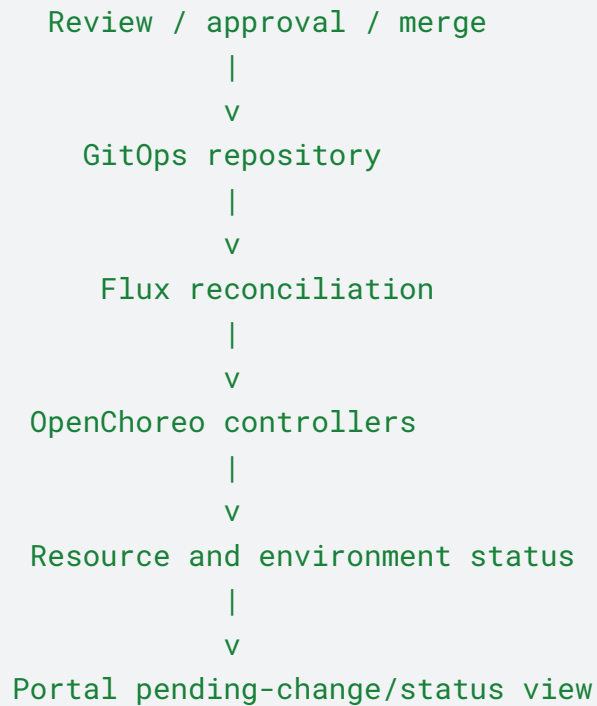
Project 702 specifically emphasizes detecting and surfacing conflicting in-flight changes rather than resolving them automatically.

15. Suggested architecture for OpenChoreo Project 702

Based on the project's proposed scope, a suitable conceptual architecture is:

None





A possible later extension would be:



This is a suggested architecture aligned with the project proposal, not a claim that every component is already implemented.

Why this design is appropriate

It preserves the separation between:

- **Developer experience:** the portal collects inputs and provides feedback.
 - **Manifest generation:** `occ` generates OpenChoreo-aware resources.
 - **Change management:** GitHub records and reviews the proposed changes.
 - **GitOps reconciliation:** Flux processes the merged desired state.
 - **Resource management:** OpenChoreo controllers reconcile the relevant resources.
 - **Status reporting:** the portal reports the progress and outcome of the change.
-

16. Important implementation questions

Before implementing a Harness-like workflow, several questions should be answered.

Questions specific to Harness

1. Is the GitOps controller configured to manage the repository and paths being updated?
2. Does the ApplicationSet Git generator match the intended configuration files?
3. Is the pipeline using Wait for PR merge or explicit Harness Approval and Merge PR steps?
4. How does the pipeline identify the Application generated by ApplicationSet?
5. Does the synchronization step target an existing Application or a newly generated one?
6. Which Datadog metric and tags are actually available?
7. What should happen when verification fails or telemetry is unavailable?

Questions specific to OpenChoreo Project 702

1. Which resource types should the first PR action support?
2. How will `occ` generate valid manifests for different GitOps repository layouts?
3. How will the action detect an open PR affecting the same resource?
4. How will the system identify a merged change that has not yet been reconciled?
5. How will the portal link a PR to the corresponding OpenChoreo resource?
6. Which source provides the authoritative status after merge: GitHub, Flux, or OpenChoreo resource conditions?
7. Should runtime health verification be included in the initial scope or implemented later?

These decisions will help keep the first implementation focused while leaving room for future enhancements.

18. Final summary

Harness GitOps PR Pipelines coordinate repository changes, pull-request review, synchronization, and optional deployment verification.

The ApplicationSet scenario works as follows:

1. Update Release Repo creates or updates environment configuration.
2. A PR is opened.
3. The PR is reviewed and merged.
4. GitOps Sync runs.
5. ApplicationSet detects matching configuration during reconciliation and generates an Argo CD Application.
6. Fetch Linked Apps retrieves information about the linked application.
7. An optional Verify step evaluates deployment metrics using a health source such as Datadog.

The central architectural principle is that **a PR merge, GitOps synchronization, and application health are separate events**. A reliable platform should report them separately rather than treating them as one successful deployment operation.

Backstage → occ → GitHub PR → review and merge → Flux reconciliation → OpenChoreo controllers → portal status reporting.